FINAL REVISION EDITION / PYTHON IMPLEMENTATIONS

TELECOMMUNICATION LAB EXAM

Complete Code & Visual Explanation Guide

Experiments 1, 6, 7 and 12. The four final, commented Python scripts you uploaded, explained with formulas, bit-by-bit examples, real program output graphs and concise viva answers.

01 PCM → **06 DSSS** → **07 FHSS** → **12 Convolutional**

> **How this pack differs from your previous 10-page preparation PDF:** it includes **every line of the final four scripts**, code-to-theory explanations, concrete numerical walk-throughs and output images generated by running your files.

What is included

Module	Study focus	Key verified result
Experiment 1 - PCM	Sampling, 8-level quantization, 3-bit words, decode + LPF	600 bits/second
Experiment 6 - DSSS	PN chips, BPSK spreading, synchronized despreading	101 recovered; 8.45 dB gain
Experiment 7 - FHSS	BPSK, 7 hop choices, mixed carrier, detection	All 10 bits recovered
Experiment 12 - codes	Two registers, XOR outputs, terminating zeros	111000010111

Recommended study order: concept diagram → key formulas → code listing → how the lines work → verified plots → viva. You can use the book as a quick refresher or learn a full experiment from scratch.

Source basis: your uploaded Exp1_PCM.py, Exp6_DSSS.py, Exp7_FHSS.py, Exp12_ConvolutionalCodes.py, and the earlier Telecom_Lab_Complete_Prep.pdf. Script listings are reproduced without changing their computational logic.

START HERE

Read the four experiments like a story

No need to memorize the entire Python syntax. Memorize the one line that performs the essential signal operation.

Experiment	Data transformation	One-line memory anchor
PCM	Analog sample → quantization index → 3 binary bits	`format(i, '03b')`
DSSS	Bit ±1 × PN chip ±1 → spread signal	`spread = message_chips * pn_chips`
FHSS	BPSK signal × selected hopping carrier	`fhss = bpsk * hop_carrier`
Convolutional	Input bit + 2 previous bits → 2 XOR outputs	`c1 = bit ^ d1 ^ d2`

Python functions you will see repeatedly

Syntax / function	What it means	Example
`np.arange()`	Generate evenly spaced numbers / indexes	`np.arange(4)` → 0,1,2,3
`np.repeat()`	Repeat each array element	[1,0] twice → 1,1,0,0
`np.tile()`	Repeat the entire pattern	[1,0] twice → 1,0,1,0
`np.cos()` / `np.sin()`	Generate carrier and message waveforms	cos(2πft)
`np.sum()`	Add all values in a region	Positive sum → decide bit 1
`plt.step()`	Draw binary / quantized stairs	0/1 codes and levels
`a ^ b`	Bitwise exclusive OR	1 ^ 1 = 0

> **Quick preparation rule:** know the **aim**, identify the **transmitter and receiver blocks**, explain the **main multiplication or XOR operation**, identify each plotted waveform and state one verified numerical result.

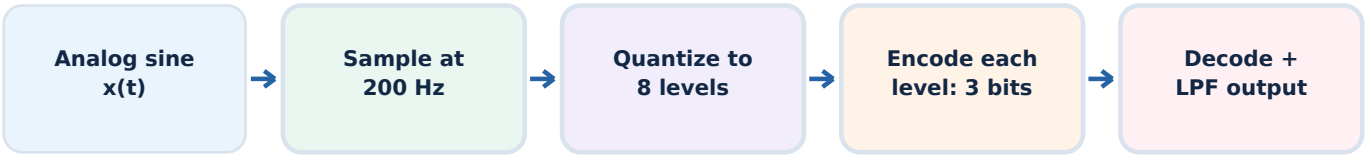
Practical-run checklist

- Use Python 3 with **NumPy** and **Matplotlib**; SciPy is optional for Experiment 1.
- Keep each .py program in its own file and run it; never confuse numeric frequencies (cycles/bit) with real radio frequencies.
- If a plot looks unfamiliar, read the subplot title before changing the code.
- For convolutional encoding, the supplied script implements an **encoder only** (not Viterbi decoding).

EXPERIMENT 01 / PCM

Analog → samples → quantized levels → bits

Aim: generate PCM modulation/demodulation and optionally reconstruct the analog message with a low-pass filter.



Core idea: sampling makes **time** discrete; quantization makes **amplitude** discrete; encoding turns each quantized level into a three-bit codeword.

Parameters in your exact script

Parameter	Used value	Why it matters
Message amplitude	A = 4	Sine spans −4 to +4 V.
Message frequency	f_m = 2 Hz	Two complete sine cycles per second.
Sampling rate	f_s = 200 Hz	Sample spacing T_s = 0.005 s.
Quantization	8 levels: −3.5, −2.5, ... +3.5	3 bits per level; step Δ = 1 V.
Time vector	t = 0, 0.005, ... 1.0	201 samples including both endpoints.

Key formulas

Message signal	$\mathbf{x(t) = 4\sin(4\pi t)}$	A sin(2π f_mt); f_m = 2 Hz.
Nyquist	$\mathbf{f_s \geq 2\, f_{max}}$	Minimum for 2 Hz is 4 Hz; this demo uses 200 Hz.
Levels	$\mathbf{L = 2^n = 8}$	n = $\log_2(8)$ = 3 bits per sample.
PCM bit rate	$\mathbf{R_b = n\, f_s}$	3 × 200 = **600 bits/s**.

Subtle exam fact: the program samples 201 endpoints, producing 603 bits in this finite array. The nominal continuous PCM rate is still 600 bits/s.

EXPERIMENT 01 / PCM

Final Python code — part 1 of 2

The next two pages reproduce the exact uploaded script. Green lines are explanatory comments; original source line numbers appear at left.

PYTHON | exact source lines 1-41

```python
"""Experiment 1: 3-bit PCM, quantization, decoding and reconstruction."""
import numpy as np
import matplotlib.pyplot as plt

# 1. Generate Original Message Signal
A, fm, fs = 4, 2, 200
t = np.arange(fs + 1) / fs
x = A * np.sin(2 * np.pi * fm * t)

# 2. Quantization (8 levels = 3 bits)
levels = np.arange(-3.5, 4, 1)
index = np.ceil(x + 4).astype(int) - 1
index = np.clip(index, 0, 7)
xq = levels[index]

# 3. PCM Encoding (Decimal to 3-bit Binary)
codes = [format(i, '03b') for i in index]
bits = [int(b) for code in codes for b in code]

# 4. PCM Decoding (Binary to Quantized Values)
decoded = np.array([levels[int(code, 2)] for code in codes])

# 5. (Optional) Signal Reconstruction using Low-Pass Filter
try:
    # Sixth-order Butterworth LPF (if SciPy is installed)
    from scipy.signal import butter, lfilter

    b, a = butter(6, 4*fm/fs)
    reconstructed = lfilter(b, a, decoded)
    print("Using Butterworth Low-Pass Filter")

except ImportError:
    # (Optional) Simple first-order LPF (without SciPy)
    alpha = 1 - np.exp(-2*np.pi*(2*fm)/fs)
    reconstructed = np.zeros_like(decoded)

    for i in range(len(decoded)):
        previous = reconstructed[i-1] if i > 0 else 0
        reconstructed[i] = alpha*decoded[i] + (1-alpha)*previous

    print("Using Simple Low-Pass Filter")
```

Read the code by operation

Lines 6–8: create a sine wave. **Lines 11–14:** select one of eight amplitude levels. **Lines 17–21:** encode three-bit words and decode back to level values.

Lines 24–41: use a sixth-order Butterworth low-pass filter when SciPy is installed; otherwise perform first-order recursive smoothing.

EXPERIMENT 01 / PCM

Final Python code — part 2 of 2

Console results and all six plots: original, sampled, quantized, PCM bits, decoded, reconstructed.

PYTHON | exact source lines 42-75

```
42
43 # 6. Display Results
44 print("First 10 PCM Codes:", codes[:10])
45 print("PCM Bit Rate:", 3*fs, "bits/s")
46
47 # 7. Plot All Waveforms
48 fig, ax = plt.subplots(3, 2, figsize=(11, 8))
49
50 ax[0, 0].plot(t, x)
51 ax[0, 0].set_title("Original Message Signal")
52
53 ax[0, 1].stem(t, x, markerfmt='.')
54 ax[0, 1].set_title("Sampled Signal")
55
56 ax[1, 0].step(t, xq)
57 ax[1, 0].set_title("Quantized Signal")
58
59 ax[1, 1].step(range(60), bits[:60])
60 ax[1, 1].set_title("PCM Encoded Bits (First 60)")
61
62 ax[2, 0].step(t, decoded)
63 ax[2, 0].set_title("Decoded Signal")
64
65 ax[2, 1].plot(t, x, 'k--', label="Original")
66 ax[2, 1].plot(t, reconstructed, label="Reconstructed")
67 ax[2, 1].legend()
68 ax[2, 1].set_title("Reconstructed Signal (LPF)")
69
70 # 8. Graph Formatting
71 for axis in ax.flat:
72     axis.grid(True)
73
74 plt.tight_layout()
75 plt.show()
```

How to interpret the unfamiliar Python syntax

Code pattern	Meaning
`np.ceil(x+4), np.clip(...)`	Round up to an interval, then keep the index in the range 0...7.
`format(i, '03b'), int(code,2)`	Encode a 3-bit binary string, then decode a binary string to decimal.
`ax[1,0], ax.flat`	Choose a subplot position and then loop through all six axes.

EXPERIMENT 01 / PCM

Work through one sample + the LPF

If you can explain this one conversion, you can explain the whole PCM encoder and decoder.

Example: a sample amplitude of +2.2 V

Analog value +2.2 → **Index = 6 (zero-based)** → **Level = +2.5 V** → **Binary PCM 110** → **Decode = +2.5 V**

Why index 6? **ceil(2.2 + 4) − 1 = 7 − 1 = 6**; `levels[6]` is +2.5 and `format(6,'03b')` is 110.

3-bit level/code table

Index	Level (V)	Code	Index	Level (V)	Code
0	−3.5	000	4	+0.5	100
1	−2.5	001	5	+1.5	101
2	−1.5	010	6	+2.5	110
3	−0.5	011	7	+3.5	111

What does the filter do?

Butterworth filter	**order = 6; cutoff = 4 Hz**	`butter(6, 4*fm/fs)` creates filter coefficients b,a.
Applying it	**y = lfilter(b,a,decoded)**	Smooths high-frequency staircase components; can create delay.
No SciPy?	**y[k] = αx[k] + (1−α)y[k−1]**	First-order fallback smooths using previous output; not identical to Butterworth.

Important distinction: decoding recovers the **quantized** staircase, while reconstruction filtering gives a **smoothed analog approximation**. The filter cannot restore amplitude precision lost during quantization.

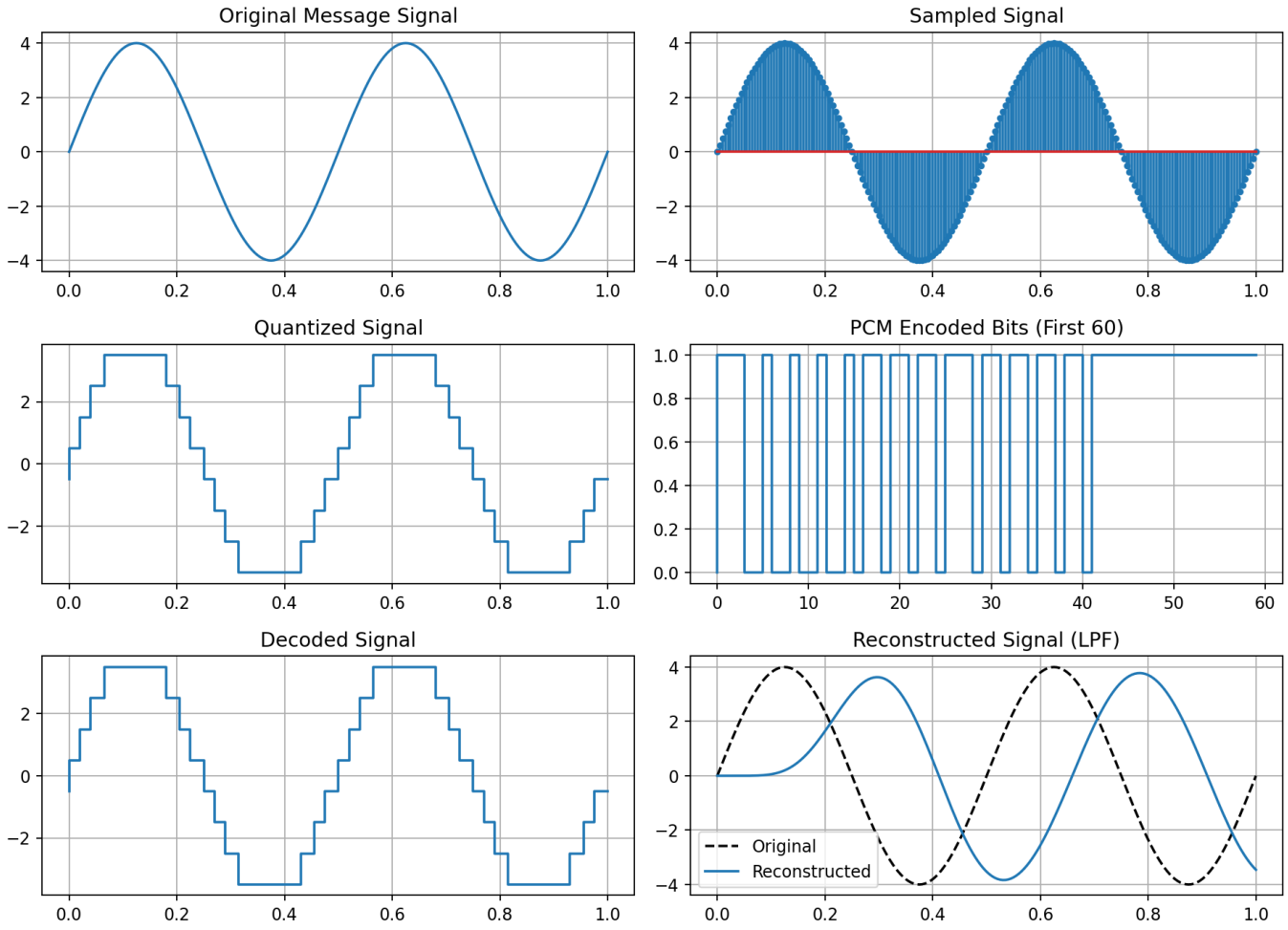
What to say when asked about the filter

“We use a low-pass filter after decoding to reduce the sharp high-frequency transitions and reconstruct a smooth version of the analog message. My Python script uses a sixth-order Butterworth LPF when SciPy is available and a simple first-order fallback otherwise.”

EXPERIMENT 01 / PCM

Verified output — six waveforms

This image is generated from the exact uploaded Exp1_PCM.py script, not a sketch.



PROGRAM OUTPUT

```
Using Butterworth Low-Pass Filter
First 10 PCM Codes: ['011','100','100','100','100','101','101','101','101','110']
PCM Bit Rate: 600 bits/s
```

Identify the six subplots in exam order

1 Original: smooth sine. **2 Sampled:** stem-style discrete samples. **3 Quantized:** eight-level staircase. **4 PCM bits:** high/low binary stream. **5 Decoded:** same quantized staircase. **6 Reconstructed:** LPF-smoothed signal; may have amplitude loss or delay.

EXPERIMENT 01 / PCM

Viva and quick checks

Short, speakable responses - the kind you can deliver in a practical viva.

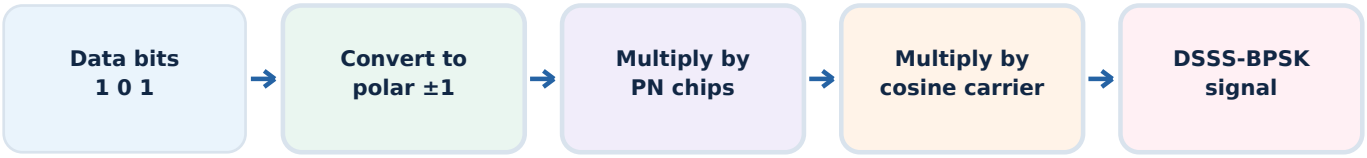
Viva question	Exam-ready short answer
What is PCM?	Pulse Code Modulation encodes quantized samples of an analog signal as binary codewords.
State the sampling theorem.	Sample at least twice the maximum signal frequency: $f_s \geq 2f_{max}$.
Why 3 bits?	There are 8 quantization levels; $\log_2(8) = 3$.
Sampling versus quantization?	Sampling makes time discrete; quantization makes amplitude discrete.
PCM bit rate?	R_b = 200 samples/s × 3 bits/sample = 600 bits/s.
What does `format(i,'03b')` do?	Converts an integer quantizer index into a three-character binary string.
Why use the LPF?	To smooth the decoded staircase toward an analog waveform.
Why reconstruction is imperfect?	Quantization error remains and the filter introduces amplitude/phase effects.
What does `try / except` do?	Tries SciPy Butterworth; if SciPy import fails, uses the first-order backup.
Why might a decoded sample at zero be −0.5?	The quantizer levels are centered at ±0.5, ±1.5, etc.; zero lies on the boundary assigned to −0.5 by this code.

Result sentence: “PCM modulation and demodulation were implemented successfully. An analog sine wave was sampled at 200 Hz, quantized to 8 levels, encoded to 3-bit words, decoded and reconstructed approximately with an LPF.”

EXPERIMENT 06 / DSSS

Spreading a data bit with PN chips

Aim: implement direct-sequence spread spectrum using a seven-chip PN code and a BPSK carrier.



Main idea: each data bit is expanded into **7 chips**. The synchronized receiver applies the same PN pattern and carrier to recover the original data.

Fixed sequences from your final script

Input	Bit pattern	Meaning
Data bits	1 0 1	Message with 3 bits.
PN bits	0 0 1 0 1 1 1	7 chips per bit, repeated 3 times.
Polar data	+1 −1 +1	`2 * data - 1`.
Polar PN	−1 −1 +1 −1 +1 +1 +1	`2 * pn - 1`.
Spread length	3 × 7 = 21 chips	40 samples/chip, therefore 840 plot samples.

Chip-by-chip spreading example

Data bit	Polar data	Spread chips (±1)
1	+1	− − + − + + +
0	−1	+ + − + − − −
1	+1	− − + − + + +

Polar mapping	$\mathbf{d = 2b - 1}$	0 becomes −1; 1 becomes +1.
Spreading	$\mathbf{s = d \times c}$	Multiply data polarity by each PN chip.
Gain	$\mathbf{G = R_c/R_b = 7}$	$10 \log_{10}(7) =$ **8.45 dB**.

EXPERIMENT 06 / DSSS

Final Python code — part 1 of 2

The bold conceptual operation is multiplication of repeated data polarities by a repeated PN code.

PYTHON | exact source lines 1-35

```python

"""Experiment 6: DSSS using 7-chip PN code and BPSK."""
import numpy as np
import matplotlib.pyplot as plt

# 1. Generate Binary Data and PN Code
data = np.array([1, 0, 1])
pn = np.array([0, 0, 1, 0, 1, 1, 1])
L, samples = len(pn), 40

# 2. Convert 0 to -1 and 1 to +1
message = 2 * data - 1
pn_polar = 2 * pn - 1

# 3. DSSS Spreading
message_chips = np.repeat(message, L)
pn_chips = np.tile(pn_polar, len(data))
spread = message_chips * pn_chips

# 4. BPSK Modulation with Cosine Carrier
baseband = np.repeat(spread, samples)
pn_wave = np.repeat(pn_chips, samples)
t = np.arange(len(baseband)) / (L * samples)
carrier = np.cos(2 * np.pi * 10 * t)
tx = baseband * carrier

# 5. Optional: Despreading and Data Recovery
despread = tx * carrier * pn_wave
received = []

for i in range(len(data)):
    start = i * L * samples
    end = (i + 1) * L * samples
    bit = int(np.sum(despread[start:end]) > 0)
    received.append(bit)
```

What the important array functions do

Expression	What it does
`np.repeat(message, L)`	Hold each polar bit sign for seven PN chips.
`np.tile(pn_polar, 3)`	Repeat the whole 7-chip PN sequence for each of the three bits.
`spread = message_chips * pn_chips`	Multiply data signs and matching PN signs chip-by-chip.

EXPERIMENT 06 / DSSS

Final Python code — part 2 of 2

The optional receiver assumes perfect carrier/PN synchronization and no channel noise.

PYTHON | exact source lines 36-60

```
36
37 # 6. Display Results
38 print("Sent bits:    ", data)
39 print("Received bits:", received)
40 print(f"Processing Gain: {10*np.log10(L):.2f} dB")
41
42 # 7. Plot DSSS Waveforms
43 fig, ax = plt.subplots(3, 1, figsize=(11, 7))
44
45 ax[0].step(t, baseband)
46 ax[0].set_title("Baseband DSSS")
47
48 ax[1].plot(t, carrier)
49 ax[1].set_title("Carrier Signal")
50
51 ax[2].plot(t, tx)
52 ax[2].set_title("DSSS-BPSK Signal")
53
54 for a in ax:
55     a.grid(True)
56     a.set_xlim(0, len(data))
57
58 ax[-1].set_xlabel("Time (bit periods)")
59 plt.tight_layout()
60 plt.show()
```

Receiver math, without the frightening notation

After BPSK modulation, the script calculates `despread = tx * carrier * pn_wave`. Because each PN sign is ±1, its square is 1. Correlating over each full data-bit interval leaves a score whose **sign** identifies the data bit.

Transmitted	**tx = d c cos(2πft)**	d = data sign; c = PN sign.
At receiver	**tx × cos × c**	d × c² × cos² = d cos².
Decision	**sum > 0 → bit 1**	Negative sum → bit 0 in this noiseless demo.

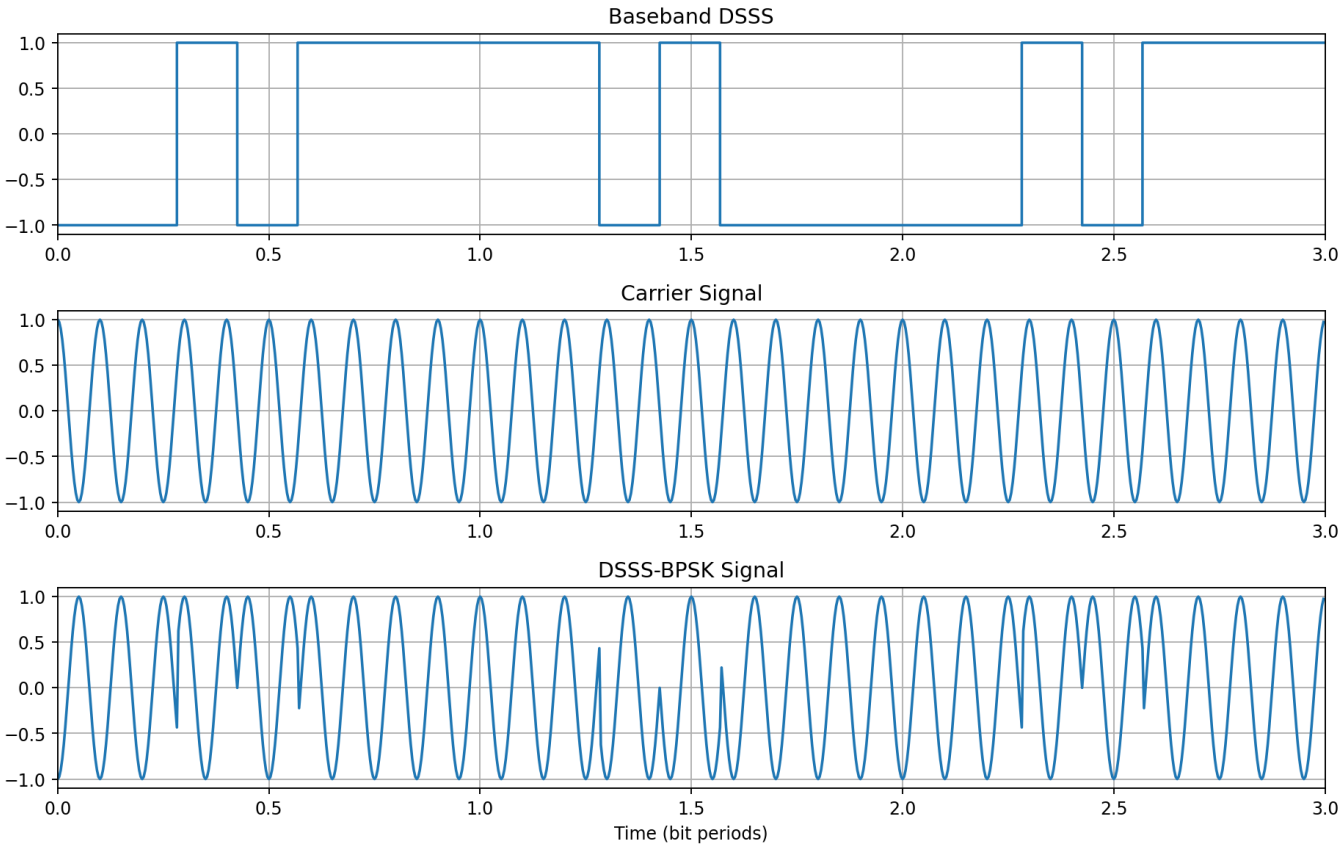
Why does the receiver select ranges of samples?

The receiver handles **7 chips × 40 samples = 280 samples** per bit. `start:end` selects that bit interval before summing to decide 1 or 0. Remember: `repeat` stretches each element; `tile` copies the whole code.

EXPERIMENT 06 / DSSS

Verified plots + viva

See baseband spreading, then the cosine carrier, then the BPSK-modulated spread waveform.



PROGRAM OUTPUT

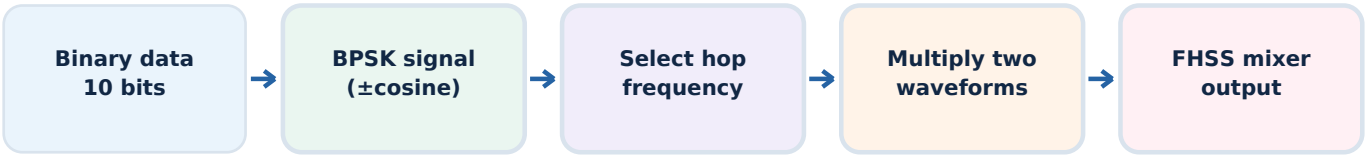
```
Sent bits:     [1 0 1]
Received bits: [1, 0, 1]
Processing Gain: 8.45 dB
```

Viva question	Exam-ready short answer
What does DSSS stand for?	Direct Sequence Spread Spectrum.
What is a chip?	One short symbol of the PN spreading code.
How many chips per bit?	Seven. Three original bits give 21 spread chips.
Why +1 and −1?	Polar signs make spreading a multiplication and produce BPSK phase inversion.
Why must the receiver PN match?	Correct despreading requires the same code at the correct timing.
What is the gain?	Spreading factor 7 or approximately 8.45 dB.

EXPERIMENT 07 / FHSS

Hopping among seven carrier frequencies

Aim: simulate FHSS by mixing a BPSK waveform with a frequency-hopping carrier.



Principle: the hop pattern selects a carrier frequency for each bit period. A synchronized receiver uses the matching hopping pattern to de-hop and detect the original bits.

One row of the hop table represents one bit period

Position	Hop index	Selected frequency
1	7	24 cycles/bit
2	3	3 cycles/bit
3	5	6 cycles/bit
4	1	1 cycle/bit
5	4	4 cycles/bit

Complete chosen carrier-frequency sequence: **24, 3, 6, 1, 4, 12, 2, 24, 2, 6 cycles/bit**. There are seven available frequencies, not seven hopping events.

Important mathematical relationships

Carrier choice	$f_{hop} = freq[hop-1]$	Hop index is one-based; Python arrays are zero-based.
BPSK waveform	$s_B = d\cos(2\pi t)$	d is +1 or −1; one base cycle per bit.
Mixer output	$s_F = s_B \times c_{hop}$	Your script directly multiplies the two waveforms.
Dwell time	T_d = 1 bit period	One hop per bit in this particular script.

Manual-to-code distinction: the manual block diagram labels M-ary FSK; the supplied numerical program and your simplified Python script instead use BPSK multiplied by a frequency-hopping carrier. This code uses a **fixed demonstration hop order**, not a new PN generator.

EXPERIMENT 07 / FHSS

Final Python code — part 1 of 2

Line numbers refer to your exact Exp7_FHSS.py upload.

PYTHON | exact source lines 1-34

```
1
2  """Experiment 7: FHSS using BPSK and seven hopping frequencies."""
3  import numpy as np
4  import matplotlib.pyplot as plt
5
6  # 1. Message bits, hop order and available frequencies
7  bits = np.array([1, 0, 1, 1, 0, 0, 1, 0, 1, 1])
8  hop_index = np.array([7, 3, 5, 1, 4, 6, 2, 7, 2, 5])
9  freq = np.array([1, 2, 3, 4, 6, 12, 24])  # cycles per bit period
10 N = 120                                    # samples per bit (one hop)
11 t_bit = np.arange(N) / N                   # time for one bit
12 t = np.arange(len(bits) * N) / N           # time for all bits
13
14 # 2. Binary Data -> Polar Bits -> BPSK Carrier
15 polar = np.repeat(2 * bits - 1, N)
16 base_carrier = np.tile(np.cos(2 * np.pi * t_bit), len(bits))
17 bpsk = polar * base_carrier
18
19 # 3. Frequency-Hopping Carrier
20 hop_carrier = np.zeros_like(t)
21
22 for i, hop in enumerate(hop_index):
23     start, end = i * N, (i + 1) * N
24     hop_carrier[start:end] = np.cos(2 * np.pi * freq[hop-1] * t_bit)
25
26 # 4. FHSS Modulation (Mix BPSK with Hopping Carrier)
27 fhss = bpsk * hop_carrier
28
29 # 5. Optional: De-hopping and Data Recovery
30 received = []
31 for i in range(len(bits)):
32     start, end = i * N, (i + 1) * N
33     score = np.sum(fhss[start:end] * hop_carrier[start:end] * base_carrier[start:end])
34     received.append(int(score > 0))
```

Five operations to recognize immediately

Python operation	Meaning
`N = 120, np.repeat(..., N)`	120 samples per bit/hop; extend the polar data for plotting.
`enumerate(hop_index)`	Get both the hop position and the selected hop number.
`freq[hop-1]`	Choose one of seven frequencies using a zero-based Python index.

EXPERIMENT 07 / FHSS

Final Python code — part 2 of 2

The program prints the selected frequency sequence and checks ideal coherent reception.

PYTHON | exact source lines 35-62

```
35
36 # 6. Display Results
37 print("Hop Frequencies:", freq[hop_index-1])
38 print("Sent Bits:      ", bits)
39 print("Recovered Bits: ", received)
40
41 # 7. Plot FHSS Waveforms
42 fig, ax = plt.subplots(4, 1, figsize=(11, 9))
43
44 ax[0].step(t, polar)
45 ax[0].set_title("Original Polar Bits")
46
47 ax[1].plot(t, bpsk)
48 ax[1].set_title("BPSK Modulated Signal")
49
50 ax[2].plot(t, hop_carrier)
51 ax[2].set_title("Hopping Carrier (7 Frequencies)")
52
53 ax[3].plot(t, fhss)
54 ax[3].set_title("FHSS Signal")
55
56 for a in ax:
57     a.grid(True)
58     a.set_xlim(0, len(bits))
59
60 ax[-1].set_xlabel("Time (bit periods)")
61 plt.tight_layout()
62 plt.show()
```

One concise example: the first hop

First bit: 1 → polar +1. **First hop index:** 7 → `freq[6]` = **24 cycles/bit**. The code generates 120 samples of that high-frequency hopping carrier and multiplies it by the BPSK waveform.

The mixer creates two frequency components

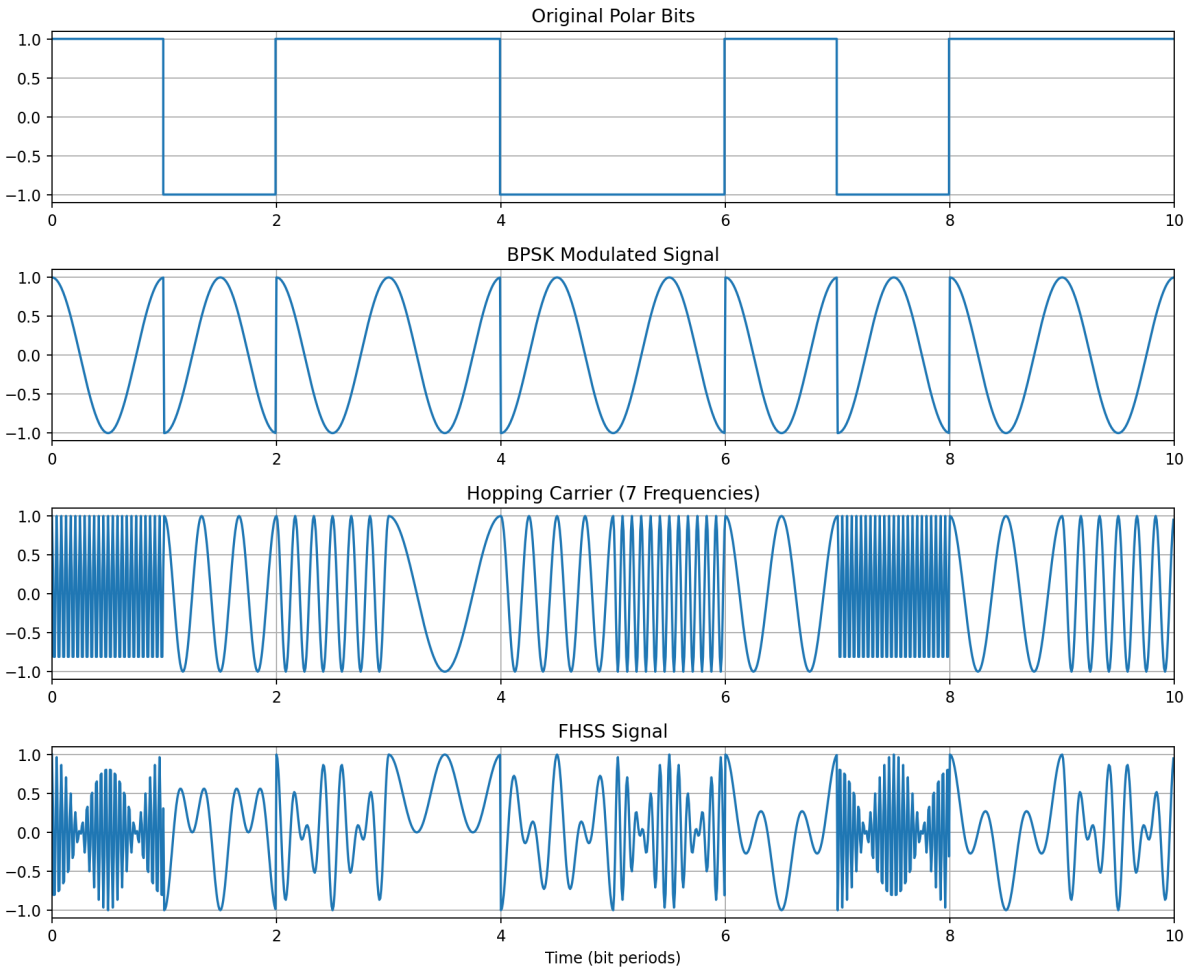
Product identity	$\cos A \cos B = \frac{1}{2}[\cos(A+B) + \cos(A-B)]$	Explains why the waveform is not simply one clean hopped sinusoid.
Receiver score	$\Sigma(s_F \times c_{hop} \times c_{base})$	Its sign reveals the original bit in a synchronized/noiseless demonstration.

Practical limitation: there is no channel noise, no hopping-timing error and no automatically generated PN sequence in this final script. These omissions are intentional simplifications for the lab demonstration.

EXPERIMENT 07 / FHSS

Verified plots + viva

Higher hop-frequency choices appear as denser, faster carrier oscillations in the third subplot.



PROGRAM OUTPUT

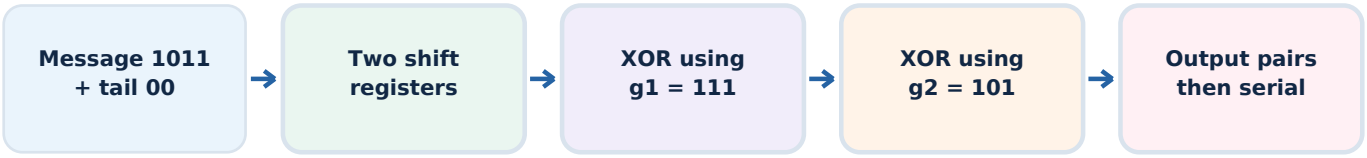
```
Hop Frequencies: [24 3 6 1 4 12 2 24 2 6]
Sent Bits:       [1 0 1 1 0 0 1 0 1 1]
Recovered Bits:  [1, 0, 1, 1, 0, 0, 1, 0, 1, 1]
```

Viva question	Exam-ready short answer
What is FHSS?	Spread-spectrum technique that changes carrier frequency according to a hopping sequence.
Why synchronization?	The receiver must use the same hop order and timing to reverse the hopping.
How many frequencies here?	Seven possible carriers, selected across ten bit periods.
DSSS versus FHSS?	DSSS multiplies by rapid PN chips; FHSS changes selected carrier frequency.
What is dwell time?	Time spent on one selected hop; one bit period in this demonstration.

EXPERIMENT 12 / CONVOLUTIONAL CODES

Memory + XOR turns one input into two outputs

Aim: implement a (2,1,2) convolutional encoder with generators 111 and 101.



Core rule: for input bit `bit` and previous-bit registers d1, d2, produce two bits using XOR, **then update the registers**.

Four XOR cases

A	B	A XOR B	Meaning
0	0	0	Same bits → 0
0	1	1	Different → 1
1	0	1	Different → 1
1	1	0	Same bits → 0

Generator equations and dimensions

First output	**$c_1 = u \oplus d1 \oplus d2$**	g_1 = 111 uses all three taps.
Second output	**$c_2 = u \oplus d2$**	g_2 = 101 skips d1.
Code rate	**R = k/n = 1/2**	One input bit creates two encoded bits.
Constraint length	**K = m + 1 = 3**	There are m = 2 memory registers.

Effective output length: four message bits plus two flush zeros → six encoder steps × two output bits = **12 transmitted bits**. The nominal coding rate is 1/2; for this short terminated frame, information fraction is 4/12 = 1/3.

EXPERIMENT 12 / CONVOLUTIONAL CODES

Final Python code — complete listing

This is the entire exact program in one readable page. It uses Matplotlib only, not NumPy or SciPy.

PYTHON | exact source lines 1-44

```python

"""Experiment 12: (2,1,2) Convolutional Encoder."""
import matplotlib.pyplot as plt

# 1. Original Message and Two Terminating Zeros
message = [1, 0, 1, 1]
input_bits = message + [0, 0]

# 2. Initialize Two Shift Registers
d1 = d2 = 0
pairs = []

# 3. Convolutional Encoding using XOR
for bit in input_bits:
    c1 = bit ^ d1 ^ d2     # Generator g1 = 111
    c2 = bit ^ d2          # Generator g2 = 101
    pairs.append([c1, c2])
    d2, d1 = d1, bit       # Shift registers

# 4. Convert Output Pairs into Serial Bit Stream
encoded = [b for pair in pairs for b in pair]

# 5. Display Results
print("Encoder input:", input_bits)
print("Output pairs:", ' '.join(''.join(map(str, p)) for p in pairs))
print("Encoded stream:", ''.join(map(str, encoded)))

# 6. Plot Original and Encoded Signals
fig, ax = plt.subplots(2, 1, figsize=(10, 5))

ax[0].step(range(len(message)+1), message + [message[-1]], where='post')
ax[0].set_title("Original Message")

ax[1].step(range(len(encoded)+1), encoded + [encoded[-1]], where='post')
ax[1].set_title("Convolutional Encoded Stream")

for a in ax:
    a.set_ylim(-0.2, 1.2)
    a.set_yticks([0, 1])
    a.grid(True)

ax[1].set_xlabel("Bit Index")
plt.tight_layout()
plt.show()
```

Four lines carry the whole algorithm

`c1 = bit ^ d1 ^ d2` and `c2 = bit ^ d2` generate the pair. `d2, d1 = d1, bit` shifts memory **after** output. The list comprehension flattens the output pairs into one stream.

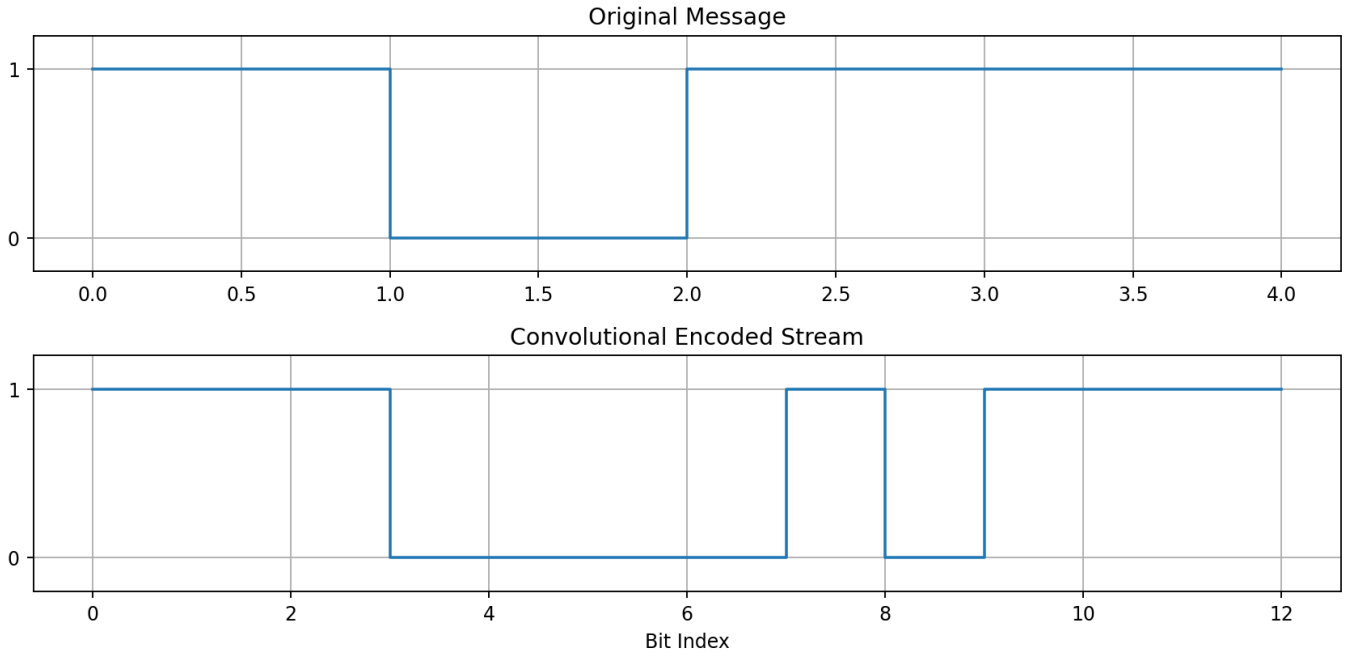
EXPERIMENT 12 / CONVOLUTIONAL CODES

Worked state table + actual encoded result

Calculate the XOR outputs from old memory first; update the registers only afterward.

Step	New bit u	Old d1	Old d2	c1	c2	Output pair
1	1	0	0	1	1	11
2	0	1	0	1	0	10
3	1	0	1	0	0	00
4	1	1	0	0	1	01
5	0 (tail)	1	1	0	1	01
6	0 (tail)	0	1	1	1	11

Memorize: message `1011` + tail `00` → **11 10 00 01 01 11** → serial output **111000010111**.




```
PROGRAM OUTPUT
Encoder input: [1, 0, 1, 1, 0, 0]
Output pairs: 11 10 00 01 01 11
Encoded stream: 111000010111
```

The first subplot plots the four original message bits; the second plots all 12 generated bits including the two tail steps. The manual displays the encoded code-vector matrix; the plotted waveforms are an added visual study aid.

EXPERIMENT 12 / CONVOLUTIONAL CODES

Viva + code explanations

Convolutional coding uses memory. Unlike PCM, there is no carrier or spreading waveform in this experiment.

Viva question	Exam-ready short answer
What is a convolutional encoder?	An error-control encoder whose output depends on current and previous input bits.
What is (2,1,2)?	Two output bits per input, one input bit at a time, two memory elements (manual notation).
Code rate?	Nominal rate is 1/2.
Constraint length?	K = 2 memory cells + 1 present input = 3.
What is XOR?	Modulo-2 addition; outputs 1 when binary inputs differ.
Why append two zeros?	Flush both memory registers back to zero at the end.
Why update `d2,d1` together?	Simultaneous assignment preserves the previous d1 before overwriting it.
What does `where='post'` mean?	Step plot holds the current bit level until the next sample boundary.
What is Viterbi decoding?	A common algorithm to estimate a likely transmitted bit path from a trellis.
Does this script decode errors?	No. It performs encoding only; Viterbi decoding is not implemented.

Result sentence for the examiner

> “A (2,1,2) convolutional encoder was implemented. XOR connections 111 and 101 produced two output bits for each input, and message 1011 with tail 00 was encoded as 111000010111.”

FINAL SYNTHESIS

Four experiments — one comparison table

Use these differences to avoid mixing up PCM, DSSS, FHSS and convolutional coding.

What?	Exp 1: PCM	Exp 6: DSSS	Exp 7: FHSS	Exp 12: code
Input	Analog sine	Binary 101	10 binary bits	Binary 1011
Main idea	Quantize then encode	PN-chip multiplication	Change hop frequencies	XOR using memory
Main operation	`format(...)`	`spread = data*PN`	`fhss = bpsk*hop`	`c1, c2 = XOR`
RX / output	Decode + LPF	Despread + correlate	Dehop + detect	Encoded pairs only
Number to say	600 bits/s	7 chips/bit	7 hop options	111000010111

Python quick dictionary

Name	Meaning	Found in
`np.ceil / np.clip`	Round up / restrict values to a valid numeric range	PCM quantizer
`format(i,'03b')`	Make a padded 3-bit binary string	PCM encoder
`int(code,2)`	Binary string to decimal integer	PCM decoder
`try / except ImportError`	Use SciPy when installed; otherwise run backup code	PCM filter
`np.repeat / np.tile`	Stretch each value / copy entire pattern	DSSS and FHSS
`enumerate()`	Return position and selected value	FHSS frequency selection
`list.append()`	Add a result to a growing list	DSSS/FHSS/encoder
`^`	XOR / modulo-2 addition on integers	Convolutional encoder

Source-aware caution: PCM in this file actually reconstructs with an LPF; DSSS/FHSS receivers are ideal noiseless demonstrations; Experiment 12 performs encoding only. Your lab manual and simplified scripts may show different optional output quantities.

LAST PAGE

15-minute last-minute recall sheet

Cover the answer column with your hand, answer aloud, then check.

Recall question	Answer you should say
PCM steps?	Sampling → quantization → binary encoding → decoding → LPF.
PCM sampling theorem?	$f_s \geq 2f_{max}$.
Why eight PCM levels?	Three bits give $2^3 = 8$ levels.
PCM output rate?	200 × 3 = 600 bits/s.
What if no SciPy?	The first-order low-pass fallback runs automatically.
DSSS polar mapping?	1 → +1 and 0 → −1.
DSSS uses which PN?	0010111, seven chips for each data bit.
DSSS processing gain?	$10 \log_{10}(7) \approx 8.45$ dB.
FHSS hop choices?	1, 2, 3, 4, 6, 12, 24 cycles per bit.
What is hop synchronization?	Transmitter and receiver select the same hop at the same time.
What are the convolutional taps?	g1 = 111, g2 = 101.
What is XOR?	Different binary inputs give 1; same inputs give 0.
Why add 00 at the end?	To flush the two memory elements.
Encoded 1011?	11 10 00 01 01 11 → 111000010111.

Suggested one-minute spoken demonstration

1. State the aim. **2.** Point to the source bits or analog message. **3.** Explain the key conversion or multiplication. **4.** Point to one output graph. **5.** Read the verified result. **6.** Explain one limitation.

> **Finish with understanding, not memorization.** If you can explain why PCM uses three bits, why DSSS uses the same PN at the receiver, why FHSS switches frequencies, and how XOR creates two convolutional output bits, you can describe all four programs confidently.

Verification: these plots and console results were generated by executing your four uploaded .py files. The displayed code pages use exact source text and source-line numbers; theoretical commentary is labeled separately when it highlights an implementation limitation or a lab-manual distinction.